Document Databases: A Deep Dive for the Relational Mind

A Comprehensive Study Guide for Computer Science Students

Transitioning from MSSQL/PostgreSQL to the Document Paradigm

April 23, 2026

Abstract

This document provides a rigorous, bottom-up exploration of document-oriented database management systems (DoDBMS) for computer science students with a strong foundation in relational databases (Microsoft SQL Server, PostgreSQL). We examine the theoretical foundations, data models, storage formats (JSON/BSON), query languages, consistency models, and distributed architectures that define the document paradigm. Rather than treating document databases as "JSON stores," we analyze them as first-class database systems with distinct theoretical underpinnings, trade-offs, and optimization strategies.

Contents

1 The Paradigm Shift: From Relations to Documents **4**
- 1.1 The Relational Foundation You Know . . . 4
- 1.2 The Document Inversion . . . 4
- 1.3 The CAP Theorem Context . . . 4

2 The Document Data Model **5**
- 2.1 What Is a Document? . . . 5
- 2.2 Collections: Schema-Less, Not Schema-Free . . . 5
- 2.3 The Object-Document Mismatch . . . 5

3 JSON and BSON: The Serialization Formats **6**
- 3.1 JSON (JavaScript Object Notation) . . . 6
  - 3.1.1 Formal Grammar . . . 6
  - 3.1.2 Type System Limitations . . . 6
- 3.2 BSON (Binary JSON) . . . 6
  - 3.2.1 Specification . . . 7
  - 3.2.2 Key Extensions Over JSON . . . 7
  - 3.2.3 Structure and Traversability . . . 7
  - 3.2.4 Space Efficiency . . . 7
- 3.3 Other Formats: UBJSON, MessagePack, CBOR . . . 7

4 Data Modeling: Rethinking Normalization **8**
4.1 The Normalization Orthodoxy . . . 8
4.2 The Document Denormalization Strategy . . . 8
4.2.1 Embedding vs. Referencing . . . 8
4.2.2 The "Building Together" Rule . . . 8
4.3 Modeling Patterns . . . 8
4.3.1 The Subset Pattern . . . 8
4.3.2 The Extended Reference Pattern . . . 9
4.3.3 The Schema Versioning Pattern . . . 9
4.3.4 The Bucket Pattern . . . 9
4.3.5 The Polymorphic Pattern . . . 9

5 Query Languages and Operators **9**
5.1 From SQL to Document Query Languages . . . 9
5.1.1 MongoDB Query API . . . 9
5.1.2 Query Operators Taxonomy . . . 10
5.2 The Aggregation Pipeline . . . 10
5.3 Map-Reduce and Alternatives . . . 11

6 Indexing Strategies **11**
6.1 The B-Tree Foundation . . . 11
6.2 Index Types . . . 11
6.3 The ESR Rule . . . 12
6.4 Index Intersection . . . 12
6.5 Covered Queries . . . 12

7 Consistency, Transactions, and ACID **12**
7.1 The BASE Paradigm . . . 12
7.2 MongoDB's Multi-Document ACID Transactions . . . 12
7.3 Read Concerns and Write Concerns . . . 13

8 Distributed Architecture **14**
8.1 Replication: Replica Sets . . . 14
8.2 Sharding: Horizontal Scaling . . . 14
8.2.1 Sharding Components . . . 14
8.2.2 Shard Keys . . . 14
8.2.3 Chunk Splitting and Balancing . . . 14
8.3 The CAP Theorem Revisited . . . 15

9 Major Document Database Implementations **15**
9.1 MongoDB . . . 15
9.2 Amazon DocumentDB . . . 15
9.3 Azure Cosmos DB (MongoDB API) . . . 15
9.4 Couchbase . . . 15
9.5 CouchDB . . . 15
9.6 Firestore . . . 15
9.7 DynamoDB (with Document Model) . . . 16
9.8 PostgreSQL JSONB . . . 16

10 Use Cases and Anti-Patterns **16**
10.1 Ideal Use Cases 16
10.2 Anti-Patterns 16

11 Migration Strategies from RDBMS **16**
11.1 Schema Discovery 16
11.2 The Migration Approaches 17
11.3 ETL Considerations 17

12 Advanced Topics **17**
12.1 Change Streams and Event Sourcing 17
12.2 Schema Validation 17
12.3 Full-Text Search 18
12.4 Time-Series Collections 18

13 Summary and Mental Model **18**

1 The Paradigm Shift: From Relations to Documents

1.1 The Relational Foundation You Know

In relational database management systems (RDBMS), data integrity is enforced through:

- **Schema rigidity**: Tables must be defined before data insertion (`CREATE TABLE`).
- **Normalization**: Data is decomposed to eliminate redundancy (1NF, 2NF, 3NF, BCNF).
- **Referential integrity**: Foreign keys enforce relationships at the storage engine level.
- **Set-theoretic algebra**: SQL is grounded in relational algebra (σ selection, π projection, $\bowtie$ join, etc.).

Your mental model is likely:

$$\text{Database} \supset \text{Schema} \supset \text{Table} \supset \text{Row} \supset \text{Column}$$

1.2 The Document Inversion

Document databases invert several of these assumptions:

Formal Definition

A **document database** is a type of NoSQL database designed to store, retrieve, and manage **semi-structured data** in the form of documents. Documents are self-describing, hierarchical data structures that do not require a predefined global schema.

The mental model becomes:

$$\text{Database} \supset \text{Collection} \supset \text{Document} \supset \text{Field} \supset \text{Value (possibly nested)}$$

From RDBMS Perspective

Think of a **collection** as a table, but one where every row can have different columns. Think of a **document** as a row, but that row can contain entire sub-tables (arrays of sub-documents) within it.

1.3 The CAP Theorem Context

Document databases typically emerge in the context of distributed systems where the CAP theorem applies:

- **Consistency**: Every read receives the most recent write or an error.
- **Availability**: Every request receives a response (without guarantee it contains the most recent write).
- **Partition Tolerance**: The system continues to operate despite network partitions.

Traditional RDBMS (single-node) choose CP. Distributed document databases often choose **AP** (Availability + Partition tolerance) with **eventual consistency**, though modern systems offer tunable consistency.

2 The Document Data Model

2.1 What Is a Document?

A document is a self-contained record that encapsulates both data *and* schema. Unlike a relational row, which derives meaning from its table definition, a document carries its own structure.

Listing 1: A MongoDB-style Document

```
{
    "_id": ObjectId("507f1f77bcf86cd799439011"),
    "title": "Deep Learning for Computer Vision",
    "authors": [
        { "name": "Ian Goodfellow", "affiliation": "Google" },
        { "name": "Yoshua Bengio", "affiliation": "U. Montreal" }
    ],
    "metadata": {
        "pages": 775,
        "publisher": "MIT Press",
        "isbn": "978-0262035613"
    },
    "tags": ["AI", "deep learning", "CNN"],
    "published": ISODate("2016-11-18T00:00:00Z"),
    "available": true,
    "borrow_count": 142
}
```

Key Insight

The document above contains scalar values (`title`), arrays (`tags`), arrays of sub-documents (`authors`), and nested documents (`metadata`). In RDBMS, representing this requires at least three tables (books, authors, metadata) with foreign keys and JOIN operations. In a document database, it is **atomic** at the document level.

2.2 Collections: Schema-Less, Not Schema-Free

A **collection** is a grouping of documents. Unlike tables:

1. Documents in a collection need not share the same fields.
2. Field types can vary across documents.
3. There is no enforced schema at the storage level (though application-level validation is common).

However, collections are **schema-flexible**, not schema-free. Applications impose implicit schemas through their query patterns.

2.3 The Object-Document Mismatch

Relational databases suffer from the "impedance mismatch":

- Object-oriented code uses inheritance, composition, and nested objects.
- RDBMS uses flat tables, requiring ORMs to map between paradigms.

Document databases align naturally with object-oriented and functional programming models because JSON maps directly to dictionaries/maps, arrays to lists, and subdocuments to nested objects.

3 JSON and BSON: The Serialization Formats

3.1 JSON (JavaScript Object Notation)

JSON is the *lingua franca* of document databases. It is a text-based, human-readable format.

3.1.1 Formal Grammar

JSON consists of:

- **Objects**: Unordered sets of key-value pairs $\{k_1 : v_1, \ldots, k_n : v_n\}$
- **Arrays**: Ordered sequences of values $[v_1, \ldots, v_n]$
- **Values**: Strings, numbers, booleans (`true`/`false`), `null`, objects, or arrays

3.1.2 Type System Limitations

JSON's type system is impoverished compared to PostgreSQL:

JSON Type	PostgreSQL Equivalent	Limitation
Number	`INTEGER`, `REAL`, `NUMERIC`	No distinction between int/float
String	`VARCHAR`, `TEXT`	No length constraints
Boolean	`BOOLEAN`	—
Null	`NULL`	Semantically ambiguous
Array	Arrays (since 9.x)	Homogeneous vs heterogeneous
Object	`JSONB`, `HSTORE`	—

From RDBMS Perspective

JSON has no native date type. Dates are serialized as ISO 8601 strings (e.g., `"2024-01-15T10:30:00Z"`), requiring application-level parsing. There is no binary data type—Base64 encoding is required, inflating size by ∼33%.

3.2 BSON (Binary JSON)

BSON is the binary serialization format used by MongoDB and others. It extends JSON with additional types and efficiency optimizations.

3.2.1 Specification

Formal Definition

BSON is a **binary-encoded serialization** of JSON-like documents. It is designed to be **traversable** (fast in-place scanning) and **efficient** (compact encoding of common types).

3.2.2 Key Extensions Over JSON

1. **ObjectId**: A 12-byte identifier (timestamp + machine ID + process ID + counter) serving as the default primary key.
2. **Date**: 64-bit integer representing milliseconds since Unix epoch.
3. **Binary**: Raw binary data with subtype annotation.
4. **Regex**: Regular expression patterns with options.
5. **Decimal128**: IEEE 754-2008 128-bit decimal floating point (financial precision).
6. **Int32/Int64**: Explicit integer types.
7. **Timestamp**: Internal MongoDB use (operation timestamps).

3.2.3 Structure and Traversability

BSON documents begin with a 32-bit integer indicating total document size. Each element is encoded as:

[1-byte type code] [C-string field name] [value]

This allows the database engine to **skip** fields without parsing them—critical for fast projection operations (`SELECT col1, col2` equivalents).

3.2.4 Space Efficiency

BSON is often *larger* than JSON for text data due to:

- Field names stored in every document (no schema to factor them out).
- Length prefixes on every element and document.

However, for numeric arrays and binary data, BSON is significantly more compact.

Key Insight

BSON's design prioritizes **read performance** and **type richness** over minimal storage footprint. This is a fundamental trade-off: RDBMS row formats (e.g., PostgreSQL's heap tuples) factor out metadata via the system catalog; BSON inlines metadata for schema flexibility.

3.3 Other Formats: UBJSON, MessagePack, CBOR

- **MessagePack**: Compact binary format, used by Redis and others.
- **CBOR** (Concise Binary Object Representation): RFC 7049, designed for IoT/constrained devices.
- **UBJSON**: Universal Binary JSON, simpler than BSON but less type-rich.

4 Data Modeling: Rethinking Normalization

4.1 The Normalization Orthodoxy

In RDBMS, you normalize to:

1. Eliminate insertion, update, and deletion anomalies.
2. Minimize storage redundancy.
3. Enforce referential integrity declaratively.

4.2 The Document Denormalization Strategy

Document databases **embrace controlled redundancy** to optimize read performance and data locality.

4.2.1 Embedding vs. Referencing

The fundamental modeling decision:

Embedding (Denormalization)	Referencing (Normalization)
Store related data in a single document	Store related data in separate documents
Single read retrieves all data	Requires application-level joins or `$lookup`
Atomic updates within document	Multi-document transactions needed for consistency
Risk of data duplication	Risk of query-time overhead

4.2.2 The "Building Together" Rule

Key Insight

Embed when:

- The embedded data is read **together** with the parent (data locality).
- The embedded data does not grow unbounded (16MB document limit in MongoDB).
- The embedded data is not updated independently across many parents.

Reference when:

- The data is updated frequently and shared across many documents.
- The relationship is many-to-many.
- The embedded array would grow beyond practical limits.

4.3 Modeling Patterns

4.3.1 The Subset Pattern

Store the most frequently accessed subset of a large document inline, with a reference to the full document.

4.3.2 The Extended Reference Pattern

Duplicate a small, stable subset of frequently-joined data (e.g., user name and avatar URL) into documents that reference that user, avoiding joins for read-heavy operations.

4.3.3 The Schema Versioning Pattern

Add a `schema_version` field to documents to allow application-level handling of schema evolution without migration downtime.

4.3.4 The Bucket Pattern

For time-series data, group measurements into "buckets" (e.g., one document per hour) rather than one document per measurement, optimizing index size and write throughput.

4.3.5 The Polymorphic Pattern

Store different entity types in the same collection, distinguished by a type discriminator field. This mirrors object-oriented inheritance and avoids sparse tables.

5 Query Languages and Operators

5.1 From SQL to Document Query Languages

Document databases do not use SQL (though some offer SQL-like interfaces). Instead, they use structured query documents.

5.1.1 MongoDB Query API

Listing 2: SQL vs. MongoDB Query Comparison

```
// SQL: SELECT * FROM users WHERE age > 21 AND city = 'NYC'
db.users.find({
    age: { $gt: 21 },
    city: "NYC"
})

// SQL: SELECT name, email FROM users WHERE age > 21
//      ORDER BY age DESC LIMIT 10 OFFSET 20
db.users.find(
    { age: { $gt: 21 } },
    { name: 1, email: 1, _id: 0 }
).sort({ age: -1 }).skip(20).limit(10)

// SQL: SELECT city, COUNT(*) FROM users GROUP BY city
//      HAVING COUNT(*) > 5
// MongoDB Aggregation:
db.users.aggregate([
    { $group: { _id: "$city", count: { $sum: 1 } } },
    { $match: { count: { $gt: 5 } } }
])
```

5.1.2 Query Operators Taxonomy

MongoDB operators follow a consistent naming convention:

Category	Operators	Description
Comparison	`$eq`, `$ne`, `$gt`, `$gte`, `$lt`, `$lte`, `$in`, `$nin`	Scalar comparisons
Logical	`$and`, `$or`, `$not`, `$nor`	Boolean composition
Element	`$exists`, `$type`	Field metadata queries
Evaluation	`$regex`, `$text`, `$where`	Pattern matching
Array	`$all`, `$elemMatch`, `$size`	Array-specific queries
Geospatial	`$near`, `$geoWithin`, `$geoIntersects`	Location queries
Bitwise	`$bitsAllSet`, `$bitsAnySet`	Low-level bit ops

5.2 The Aggregation Pipeline

MongoDB's aggregation framework is a **data processing pipeline** modeled after Unix pipes. Each stage transforms the document stream.

Listing 3: Aggregation Pipeline Example

```
db.orders.aggregate([
    // Stage 1: Match (filter) - like WHERE
    { $match: { status: "shipped", year: 2024 } },

    // Stage 2: Lookup (left outer join)
    { $lookup: {
        from: "customers",
        localField: "customerId",
        foreignField: "_id",
        as: "customer"
    }},

    // Stage 3: Unwind (flatten array)
    { $unwind: "$customer" },

    // Stage 4: Group (aggregation)
    { $group: {
        _id: "$customer.region",
        totalRevenue: { $sum: "$amount" },
        avgOrder: { $avg: "$amount" },
        orderCount: { $sum: 1 }
    }},

    // Stage 5: Sort
    { $sort: { totalRevenue: -1 } },

    // Stage 6: Project (reshape output)
    { $project: {
        region: "$_id",
        totalRevenue: 1,
        avgOrder: { $round: ["$avgOrder", 2] },
```

```
        orderCount: 1,
        _id: 0
    }}
])
```

From RDBMS Perspective

The aggregation pipeline is MongoDB's answer to SQL's `SELECT ... FROM ... JOIN ... WHERE ... GROUP BY ... HAVING ... ORDER BY`. However, because stages are ordered and stateful, you must consider **pipeline optimization**: placing `$match` early reduces document flow to subsequent stages, just as predicate pushdown works in query planners.

5.3 Map-Reduce and Alternatives

MongoDB historically supported Map-Reduce, but it is deprecated in favor of the aggregation pipeline. CouchDB still uses Map-Reduce (JavaScript views) as its primary query mechanism.

6 Indexing Strategies

6.1 The B-Tree Foundation

Like PostgreSQL and MSSQL, MongoDB uses **B-Trees** (specifically B+ Trees) for standard indexes. The principles of index selectivity, cardinality, and covering indexes apply identically.

6.2 Index Types

Index Type	Description
Single Field	Index on one field. Supports equality and range queries.
Compound	Index on multiple fields. Order matters (left-prefix rule applies).
Multikey	Index on array fields. Creates index entries for each array element.
Text	Inverted index for full-text search across string content.
Geospatial	2dsphere (GeoJSON) and 2d (legacy coordinate pairs) indexes.
Hashed	Hash of field value. Used for hashed sharding, not range queries.
Wildcard	Indexes all fields/subfields matching a pattern (MongoDB 4.2+).

6.3 The ESR Rule

For compound indexes in MongoDB, the **Equality, Sort, Range** rule optimizes performance:

1. **Equality** fields first (exact matches).
2. **Sort** fields second (ordering).
3. **Range** fields last (inequalities, `$in`, etc.).

This mirrors the RDBMS wisdom of placing equality predicates before range predicates in composite indexes.

6.4 Index Intersection

MongoDB can use multiple indexes to fulfill a query (index intersection), but it is generally less efficient than a single compound index—similar to RDBMS behavior.

6.5 Covered Queries

A query is **covered** when all fields in the query predicate and projection are contained in the index. MongoDB can then satisfy the query entirely from the index without fetching documents (an "index-only scan"), analogous to PostgreSQL's `Index Only Scan`.

7 Consistency, Transactions, and ACID

7.1 The BASE Paradigm

Document databases historically prioritized availability over consistency:

Formal Definition

BASE stands for:

- **Basically Available**: The system guarantees availability.
- **Soft state**: State may change over time without input (eventual consistency).
- **Eventually consistent**: If no new updates are made, reads will eventually reflect the latest write.

7.2 MongoDB's Multi-Document ACID Transactions

Since MongoDB 4.0 (replica sets) and 4.2 (sharded clusters), multi-document ACID transactions are supported:

- **Atomicity**: All operations in a transaction commit or abort together.
- **Consistency**: Constraints and rules are maintained (though document DBs have fewer declarative constraints).
- **Isolation**: Snapshot isolation (reads see a consistent snapshot from transaction start).
- **Durability**: Journaled writes ensure persistence.

Listing 4: Multi-Document Transaction

```
const session = db.getMongo().startSession();
session.startTransaction();
try {
    const accounts = session.getDatabase("bank").accounts;
    accounts.updateOne(
        { _id: "alice" },
        { $inc: { balance: -100 } }
    );
    accounts.updateOne(
        { _id: "bob" },
        { $inc: { balance: 100 } }
    );
    session.commitTransaction();
} catch (error) {
    session.abortTransaction();
    throw error;
} finally {
    session.endSession();
}
```

From RDBMS Perspective

While transactions exist, they carry performance overhead in distributed document databases. The design philosophy remains: **model your data so that related data lives in a single document**, making atomic single-document updates sufficient for most operations.

7.3 Read Concerns and Write Concerns

MongoDB allows tuning consistency per operation:

Write Concern	Behavior	Analogy
`{w: 0}`	Fire-and-forget (unacknowledged)	Async, risky
`{w: 1}`	Acknowledged by primary	Default, safe
`{w: "majority"}`	Committed to majority of nodes	Strong durability
`{j: true}`	Journaled to disk	fsync equivalent

Read Concern	Behavior	Consistency
`"local"`	Read from local node (may rollback)	Fastest, weakest
`"available"`	Similar to local for sharded clusters	Fast
`"majority"`	Only committed data	Strong consistency
`"snapshot"`	For transactions, consistent snapshot	Serializable view
`"linearizable"`	Linearizable reads (slowest)	Strict consistency

8 Distributed Architecture

8.1 Replication: Replica Sets

MongoDB uses **replica sets** for high availability:

- **Primary**: Accepts all write operations.
- **Secondaries**: Replicate the primary's oplog (operation log) asynchronously.
- **Arbiter**: Participates in elections without storing data.

If the primary fails, secondaries hold an election (Raft-like consensus) to promote a new primary.

Key Insight

The **oplog** (operations log) is a capped collection that records all write operations. Secondaries tail the oplog to apply changes. This is conceptually similar to PostgreSQL's WAL (Write-Ahead Log) streaming replication, but the oplog is at the **logical** operation level rather than the physical page level.

8.2 Sharding: Horizontal Scaling

When data exceeds the capacity of a single node, **sharding** distributes data across multiple servers.

8.2.1 Sharding Components

- **Shard**: A replica set holding a subset of data.
- **Config Servers**: Store metadata about which chunks live on which shards.
- **Mongos Router**: Query router that directs operations to the correct shard(s).

8.2.2 Shard Keys

Choosing a **shard key** is the most critical design decision:

- **Cardinality**: High cardinality prevents chunk imbalance.
- **Frequency**: Low-frequency values prevent "hot shards."
- **Monotonicity**: Monotonically increasing keys (e.g., timestamps, ObjectIds) cause write hotspots on the last shard.

8.2.3 Chunk Splitting and Balancing

Data is divided into **chunks** (default 64MB). The balancer migrates chunks between shards to maintain even distribution. This is analogous to range partitioning in PostgreSQL, but automated and distributed.

8.3 The CAP Theorem Revisited

System	Choice	Notes
MongoDB (single node)	CP	Like any single-server RDBMS
MongoDB (replica set, default)	AP	Reads from secondaries may be stale
MongoDB (majority reads/writes)	CP	Strong consistency, lower availability
CouchDB	AP	Multi-version concurrency, conflict resolution
DynamoDB	AP (tunable)	Per-operation consistency settings

9 Major Document Database Implementations

9.1 MongoDB

The dominant document database. Features:

- BSON storage, rich query language, aggregation pipeline.
- Multi-document ACID transactions.
- Change streams (reactive programming, event sourcing).
- Time-series collections, clustered indexes, columnar compression (5.0+).
- Atlas: Managed cloud service with global clusters.

9.2 Amazon DocumentDB

AWS-managed service with MongoDB compatibility (3.6/4.0/5.0 API). Uses Aurora's distributed storage subsystem rather than MongoDB's storage engine.

9.3 Azure Cosmos DB (MongoDB API)

Microsoft's globally distributed database with multiple API choices (SQL, MongoDB, Cassandra, Gremlin). Offers tunable consistency levels and multi-region writes.

9.4 Couchbase

Combines document storage with memcached caching. Uses N1QL (SQL-like query language for JSON). Strong focus on mobile/edge synchronization (Couchbase Lite).

9.5 CouchDB

Apache project emphasizing REST API, eventual consistency, and multi-master replication. Uses Map-Reduce views and Mango queries.

9.6 Firestore

Google's serverless document database. Real-time synchronization, offline persistence, and hierarchical collections. Optimized for mobile and web applications.

9.7 DynamoDB (with Document Model)

While primarily a key-value store, DynamoDB supports document-like items (nested maps/lists). Single-table design patterns enable complex access patterns.

9.8 PostgreSQL JSONB

Not a document database per se, but PostgreSQL's `JSONB` type allows hybrid relational/document workloads. Supports JSON indexing (GIN indexes), JSON aggregation, and SQL/JSON path queries (SQL:2016 standard).

From RDBMS Perspective

If you love PostgreSQL but need document flexibility, `JSONB` is a powerful middle ground. You get ACID transactions, joins, and relational constraints alongside schema-flexible JSON columns. However, you lose some document-native features like horizontal sharding and flexible replication topologies.

10 Use Cases and Anti-Patterns

10.1 Ideal Use Cases

1. **Content Management**: Articles, blogs, product catalogs with varying attributes.
2. **User Profiles**: Heterogeneous user data, preferences, activity logs.
3. **IoT and Time-Series**: Sensor data with the bucket pattern.
4. **Mobile Applications**: Offline-first sync (Couchbase, Firestore).
5. **Real-Time Analytics**: Aggregation pipelines on event streams.
6. **E-Commerce**: Product catalogs with dynamic attributes (size, color, specs vary by category).

10.2 Anti-Patterns

1. **Massive Arrays**: Unbounded arrays cause document bloat and performance degradation.
2. **Deep Nesting**: Excessively nested documents complicate querying and indexing.
3. **Data Duplication Without Bound**: Updating duplicated data across many documents becomes expensive.
4. **Missing Indexes**: Collection scans on large datasets are catastrophic (table scans).
5. **Monotonic Shard Keys**: Causes write hotspots.
6. **Treating it like a Relational Database**: Excessive normalization and application-level joins defeat the purpose.

11 Migration Strategies from RDBMS

11.1 Schema Discovery

Analyze existing tables, relationships, and query patterns. Identify:

- **Master-detail relationships** suitable for embedding.

- **Many-to-many relationships** requiring referencing.
- **Read-heavy vs. write-heavy** access patterns.

11.2 The Migration Approaches

1. **Big Bang**: Migrate all at once. High risk, requires downtime.
2. **Incremental**: Migrate entity types one at a time. Dual-write period.
3. **Strangler Fig Pattern**: Gradually replace RDBMS functionality. New features use document DB; old features migrate over time.
4. **Polyglot Persistence**: Keep RDBMS for transactional data; use document DB for content/search data.

11.3 ETL Considerations

- Handle type mismatches (dates, decimals, binary).
- Flatten or nest based on access patterns, not source schema.
- Rebuild indexes after bulk load for efficiency.
- Validate data integrity post-migration (counts, checksums, spot checks).

12 Advanced Topics

12.1 Change Streams and Event Sourcing

MongoDB change streams provide a pub/sub interface to the oplog:

```
const changeStream = db.users.watch();
changeStream.on("change", next => {
    printjson(next);
    // next.operationType: "insert", "update", "delete", etc.
    // next.fullDocument: the post-change document
});
```

This enables event-driven architectures, materialized view updates, and cache invalidation.

12.2 Schema Validation

MongoDB supports JSON Schema validation at the collection level:

```
db.createCollection("students", {
    validator: {
        $jsonSchema: {
            bsonType: "object",
            required: ["name", "gpa"],
            properties: {
                name: { bsonType: "string" },
                gpa: { bsonType: "double", minimum: 0, maximum:
                    4.0 }
            }
        }
    },
```

```
    validationLevel: "strict",
    validationAction: "error"
})
```

This provides a middle ground between schema-less flexibility and RDBMS rigor.

12.3 Full-Text Search

MongoDB Atlas offers Atlas Search (based on Apache Lucene). Self-managed MongoDB supports text indexes with basic stemming and relevance scoring.

12.4 Time-Series Collections

MongoDB 5.0+ optimizes storage for time-series data:

- Automatic bucketing of measurements.
- Columnar compression for numeric arrays.
- Reduced index overhead.

13 Summary and Mental Model

Key Insight

Your new mental model:

1. **Documents are atomic units of consistency**. Design around them.
2. **Embedding is the default**; referencing is the exception.
3. **Query patterns drive data model**, not the other way around.
4. **Indexes are not automatic**; you must design them like in RDBMS.
5. **Distributed by design**; think about replication and sharding early.
6. **Schema flexibility is a feature, not an excuse for chaos**. Use validation.

"The question is not whether document databases are better than relational databases. The question is which data model best fits your access patterns, consistency requirements, and operational constraints."